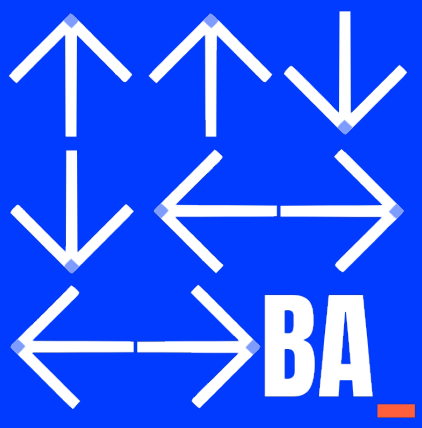
{EPITECH}

CARTRIDGE_

< 1, 2, 3, [OUT OF MEMORY] />



CARTRIDGE

binary name: NAMEOFYOURGAME.gb
language: C or Assembly
compilation: Makefile
Authorized tools: GBDK 2020, RGBDS

Gameboy

Your goal is to code a gameboy game.

Your game should be collection of a few mini-games. Think about Mario Party, carnivals, flash games... This type of game has to be well optimised in order to feel nice to play, which will give you multiple opportunities to use graphical and technical tricks in order to overcome the systems' low capacities.

Features

Gameboy System

The term “Gameboy” can refer to a multitude of consoles. In this project, you will be tested on a **Gameboy (DMG-001)** emulator.

This means that for this project, you will work with very **limited hardware resources**.
Here are the Gameboy specs:
- CPU speed (4.19 MHz)(with additional limitations) and instruction set (Sharp LR35902)
- Limited RAM (8KB) and ROM.
- Limited color palette (4 colors per 8x8 tile).
- Sprite limitations (max 40 sprites, 10 per scanline).
- Sound: 4 channels (2 pulse, 1 wave, 1 noise).
- Input: 4 directions + A/B + Start/Select.

Each of those limitations will become an obstacle we want you to overcome in a **creative way**. Work with the system, AND against.

Emulator

This project will be tested on the **Retroarch Emulator**, with the preset core **Gambatte**. It is cross-platform and open-source, with a good set of features and options.

🚀 Your program should work on most emulators tho. You can try some mobile ones, some online ones, or maybe even a real gameboy...

Libraries and Tools

You are free to use any asset or asset editor you’d like, as long as they don’t handle code logic. You can use the **GBDK library and / or the RGBDS** (including hardware.inc) to code your game. Your virtual cartridge should simulate having a **Memory Bank Controller of type MBC5**.

The time you have for this project’s realization is quite short, and might involve a few repetitive tasks. You will not be evaluated on tools you might want to create to help your tasks, however you could be rewarded if they allow you to increase the quality of your final game.

Screen & Graphics

Your game must be **beautiful**

At all times during a level, you should make use of **both the background and the window layer, and have additional sprites.** This can be as simple as animating a few tiles on each, and as complex as creating a parallax effect created via mid-frame scanline interruption.

Here are a few features of games that should be present in yours and where you should use polished graphics in order to demo of your skills:
- Menus
- Transitions (at level select for exemple)
- The games themselves
- Animations
- ...

Mini-Games

We require as many mini-games as there are people in your group.

Those mini-game should not be too big: We are not looking for lore, world building, e-sport potential, emotional cutscenes, innovative features or cutting-edge tech.

However, they must'nt be too small either. You wouldn't play pong for more than a few minutes in this day and age, at least not the OG one. Why is that? - Find hooks, find a way for your game to stay fun after a long time playing, craft your gameplay loop, follow the fun... Visual improvements are important too (see above) : your games should look and be polished, cool, animated and coherent.

If you need inspiration, here is a list of games that would fit this scope perfectly. Picking a game from this list and reproducing it is fine (if you're in good faith), this is not a game design project : Any game from the Arcade project, froggers, brickbreaker, a deterministic vampire survivor or other bullet-hell type of game, any mini-game from neopet, the dino game from google if you added a few power ups, flappy bird if you added a few power-ups, a multi-step recipe from cooking-mama...

In case you WANT to do some game design, here is a starting point.

> But don't spend too much time there, again, this is not a game design project

Each game also must have their own **leaderboard** with information saved on the cartridge.

Technical Guidelines

Architecture

More than ever, modularity is going to be very important in this project: Since your ressources are limited, you can't afford to have code duplicates. But on the other hand, hardcoding values is going to be an appropriate solution in some cases. Thread this fine line and focus on efficiency. Coherent modules (display, logic, inputs, assets...) will help you. We give you the possibility to use two libraries for this project: Are there features from the one you didn't pick that would make sense for you to implement ?

Ressource Management

The main aspect of this project is going to be the fact that you'll be limited in ways you weren't before. This means that some solutions or reflexes won't be applicable in this context. However, ingeniosity and ”over-the-top” optimisations will become worth it. Those can be purely technical, exploiting bugs in the hardware or weird quirks, be visual tricks that border on optical illusions... Look up what ”isn't possible” and you'll find explanations from people who did it. We want you to try some of those techniques.

Stability mustn't be sacrificed. Memory leaks will be deadly for your project, crashes are as unacceptable as ever, bugs are still problematic...

Identify bottlenecks and heavy ressources, and find ways to correct them. Document your process, it will be useful for your defense.

Documentation

Most of the quality assurance principles you apply to your other projects will apply here too. We expect clear technical documentation (README, architecture diagrams, anything that would help future developers on the project...)

Bonuses

As usual with game programmation, we will try to fit as many features in the main grading scale, but here are a few features that count directly towards bonuses:
• Adding a gameboy color custom palette to take advantage of its retro-compatibility capacities
• Flashing a real gameboy cartridge and testing on a real gameboy
• Creating a peripheral for the gameboy that adds buttons that your game recognizes

As always, have fun!

v 0.2

{EPITECH}